

1. Title

Task-Free Online LoRA with Drift Detection and Inference-Time Routing for Continual Instruction Tuning

2. Abstract

Large language models deployed on non-stationary instruction streams need continual updates without task boundaries, replay-heavy storage, or full-model retraining. The current lora-code repository now implements a task-free online LoRA system built around anchor-based drift detection, a growing LoRA bank inference-time routing, and orthogonal-weight anti-overlap regularization with activation-space monitoring. Relative to RP(Lora)_v2, this v3 report updates the proposal into a code-grounded paper-style status document: drift is monitored with teacher-forced answer NLL on core/probe anchors under a calibrated one-sided CUSUM rule; branch allocation is handled by a LoRA bank that freezes prior adapters and spawns new branches on drift; router pseudo-labels are produced from per-branch answer NLL with margin filtering; and the current implementation trains the active branch segment-wise while using the router for per-example selection at evaluation time. On the currently completed single-seed InstrDialog runs, the current main-table Ours configuration (the no-overlap winner) reaches `seen_avg` 0.0421, `token_f1` 0.0411, and `oracle_agreement` 0.0806. It improves on the best completed InstrDialog baseline in `seen_avg` (PeriodicLatest: 0.0263), but it does not beat the strongest baseline on `token_f1` (PeriodicLatest: 0.1552) or forgetting (Replay(50): 0.0222). Cross-benchmark coverage remains incomplete because the current main matrix still lacks several InstrDialog++ routed counterparts, including the full Ours row. The updated report therefore presents both the implemented method and the most important remaining risks before a paper-ready claim set is possible.

3. Background and Motivation

3.1 Real-world problem: non-stationary instruction streams

The repository is organized around continual instruction tuning on segmented CITB streams. New instruction data arrives sequentially, domains and response patterns change over time, and explicit task boundaries are not available at inference time. This makes the setting closer to online adaptation than to classical multi-task fine-tuning.

3.2 Why naive fine-tuning fails: catastrophic forgetting

Sequential LoRA remains a meaningful baseline in this codebase, but the current results still show that purely single-branch adaptation can preserve neither seen-average performance nor generation quality when the stream shifts. Replay helps forgetting, but it does so under explicit memory budgets and does not solve the task-free inference problem.

3.3 Why parameter-efficient adaptation (LoRA) is a good fit

The implementation keeps the backbone frozen and manipulates only LoRA adapters, which makes branch creation, freezing, and selective activation operationally cheap. This is what allows the bank-based design to stay practical inside a single training loop in `core/train.py`.

4. Related Work

The code and RP remain anchored in four literatures: LoRA / PEFT for efficient adaptation, continual instruction tuning benchmarks such as CITB, LoRA-bank and routing approaches for expert specialization, and task-free online continual learning based on change-point or dynamics-driven drift detection. The updated code path is closest in spirit to a task-free LoRA bank with explicit anchor monitoring, but it is more conservative than the original RP in what is actually implemented today.

5. Research Gap and Updated Questions

5.1 Gap

Compared with RP(Lora)_v2, the main gap has shifted. The repository no longer asks whether drift detection, branch allocation, routing, and overlap control can be wired together at all; that integration now exists. The remaining gap is whether the current implementation delivers a robust trade-off across seen-average performance, forgetting, routing quality, and generation behavior, while also generalizing beyond a partially completed single-seed result matrix.

5.2 Updated questions

- Can anchor-based drift monitoring improve seen-average performance without making the detector too conservative or too noisy?
- Does a bank of frozen / active LoRA branches help enough to justify the extra routing and monitoring complexity?
- Does the current router improve downstream performance beyond standalone routing agreement statistics?
- How much of the remaining failure profile is caused by continual adaptation, and how much by open-loop sequence behavior?

6. Proposed Method

6.1 Overview

The current implementation uses a single frozen Llama-3.1-8B-Instruct backbone and a configurable set of modules: drift detector, LoRA bank, router, and overlap loss. The bank is initialized with branch b0; subsequent branches are created only through the bank manager. Evaluation becomes task-free by selecting a branch for each prompt when both the router and bank are enabled.

6.2 LoRA parameterization (core formula)

The code follows standard LoRA parameterization: $\Delta W = BA$ with rank r , frozen base weights W , and trainable low-rank matrices A and B only. The wrapper in `core/models/lora_wrapper.py` exposes multi-adapter creation, freezing, and adapter-wise optimizer stepping so that the rest of the training loop can stay unified across baselines and our method.

6.3 Drift detection via anchor monitoring + curriculum-aware sequential change statistics

Anchors are constructed from eval examples across stream segments and split into core and probe subsets using K-Center Greedy feature sampling based on model activations, ensuring a diverse and representative anchor set. For each monitoring event, the model computes teacher-forced answer NLL on core and probe anchors. The detector smooths both streams with EMA, calibrates baselines and standard deviations over an initial window, and then applies a one-sided CUSUM-style update. A drift event is triggered only when probe degradation is sustained and the core subset also passes a guard condition. When the trigger fires, the active branch can be frozen, a new branch is spawned, and the anchor set is refreshed. Implementation note: the current code path performs anchor monitoring once per segment after evaluation, not every K optimization steps. The `monitor_interval` field is part of the detector configuration, but the operational monitoring schedule is segment-level in `run_ours(...)`. The reported drift quality is also a proxy metric because every inter-segment boundary is treated as a candidate shift.

6.4 Unified LoRA bank management + inference-time routing (closed-loop)

The LoRA bank tracks branch metadata, the active branch, frozen branches, and the maximum branch budget. When the capacity limit is reached, a LoRA Branch Merging mechanism is triggered, which performs weighted averaging of the two most similar frozen branches to free up capacity. The router is a lightweight linear classifier over pooled prompt activations. During router training, the code scores each training example under every branch, uses the minimum answer NLL as a pseudo-label, and applies Dynamic Margin Filtering (where the margin linearly increases from 0.0 to 0.02 based on the number of updates) to ensure stable pseudo-label training. During evaluation, the router extracts prompt features, predicts a branch, switches the active adapter, and then generates with that branch. Implementation note: RP(Lora)_v2 described a stronger online loop in which routing would directly determine the branch used for each training example. The current repository does not yet do that in `run_ours(...)`. Instead, segment training updates only the current active branch, and router supervision is applied afterward using pseudo-labels from the trained bank. Per-example routing is therefore a real evaluation-time behavior, but not yet the main training-time update rule.

6.5 Anti-overlap regularization for LoRA specialization (Orthogonal Weight Regularization)

The repository implements anti-overlap as Orthogonal Weight Regularization, penalizing the pairwise cosine similarity of the flattened LoRA weight matrices between different branches. This directly encourages functional specialization of LoRA branches in the parameter space. There is also a logged post-segment overlap proxy computed on anchor or eval prompts to monitor activation-space diversity.

7. Experimental Evaluation

7.1 Benchmark and setup

The current matrix is built around CITB instrdialog and instrdialog++ with Llama-3.1-8B-Instruct as the backbone. Replay budgets follow the planned 0/10/50 pattern, routing is hard top-1, and the main summary is still single-seed on the completed runs. At the time of this v3 build, the refreshed artifact summary reports 15 available runs and 1 missing run across the current main+ablation package.

7.2 Baselines

- Sequential LoRA (single branch, no drift, no router)
- Replay LoRA with memory budgets 10 and 50
- Periodic Multi-LoRA (latest-only inference)
- Bank + no router (drift + bank, latest-only inference)
- Router-only (fixed branches with routing, no drift detector)
- Ours-family variants (drift + bank + router; the current main-table winner is the no-overlap configuration, while OursFull keeps anti-overlap enabled in the ablation table)

7.3.1 Metrics

- Final-step performance: `eval.current_score`
- Seen-average performance / anytime score: `eval.seen_avg_score`, `eval.anytime_score`
- Forgetting: `eval.forgetting`
- Open-loop quality diagnostics: `eval.token_f1_mean`, `eval.lcs_overlap_mean`, prefix matches
- Routing quality: oracle agreement, decision confidence, entropy, oracle margin, branch utilization
- Drift proxy quality: false-alarm rate, miss rate, detection delay mean
- Specialization proxy: `overlap_mean_cosine`

7.3.2 Figures

The current figure set is generated by `scripts/build_paper_artifacts.py` and reuses the RP(Lora)_v2 numbering. To keep this draft focused on textual and numerical alignment, the figure assets are not embedded below; this section records the intended figure content only. Figure 3 currently emphasizes memory-budget performance; branch-budget sweeps remain available in the paper matrix configs but are not yet separated into a standalone panel in the generated figure set.

Figure 1 would show anytime performance over seen segments for the currently available main runs.

Figure 2 would summarize mean forgetting across the currently available main runs.

Figure 3 would compare performance against memory budget on the currently available matrix rows.

Figure 4 would visualize drift diagnostics from the current main-table Ours row on InstrDialog.

Figure 5 would compare router-oracle agreement and branch utilization for the currently available routed runs.

Figure 6 would depict the current closed-loop pipeline schematic used in the artifact builder.

7.4 Main Results

On instrdialog, the current main-table Ours configuration (the no-overlap winner) is currently the best completed method on seen-average performance (0.0421), ahead of PeriodicLatest at 0.0263. It does not beat the strongest baseline on token F1 (PeriodicLatest: 0.1552) or forgetting (Replay(50): 0.0222). The current Ours row is reported from variant ours_no_overlap rather than OursFull, which matters when interpreting the overlap claim. RouterOnly reaches the highest completed oracle agreement (0.7204) but does not translate that into downstream accuracy, which suggests that branch selection quality alone is not sufficient. On instrdialog++, the currently completed main-table rows are BankNoRouter, PeriodicLatest, Replay(10), Replay(50). Among those available rows, BankNoRouter reaches final-step 0.1000, seen_avg 0.1026, and forgetting 0.2756. RouterOnly is still listed as missing/running in the current summary, and the routed Ours row is not yet available, so the repository does not yet support cross-benchmark claims about the routed method.

Table 1. Main comparison on the currently completed matrix rows.

benchmark	method	final_step	seen_avg	forgetting	token_f1	oracle_agreement
instrdialog	Sequential	0.0000	0.0000	0.0389	0.0000	0.0000
instrdialog	Replay(10)	0.0000	0.0000	0.0778	0.0654	0.0000
instrdialog	Replay(50)	0.0000	0.0000	0.0222	0.0438	0.0000
instrdialog	PeriodicLatest	0.0000	0.0263	0.3380	0.1552	0.0000
instrdialog	BankNoRouter	0.1000	0.0211	0.1870	0.1247	0.0000
instrdialog	RouterOnly	0.0000	0.0000	0.0556	0.0731	0.7204
instrdialog	Ours	0.0000	0.0421	0.0824	0.0411	0.0806
instrdialog++	Sequential	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
instrdialog++	Replay(10)	0.0000	0.0000	0.0270	0.0429	0.0000
instrdialog++	Replay(50)	0.0000	0.0000	0.0198	0.0343	0.0000
instrdialog++	PeriodicLatest	0.0000	0.0100	0.2008	0.1353	0.0000
instrdialog++	BankNoRouter	0.1000	0.1026	0.2756	0.1545	0.0000
instrdialog++	RouterOnly	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
instrdialog++	Ours	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

7.5 Ablations and diagnostics

Compared with the current main-table Ours row (ours_no_overlap), removing the router lowers seen_avg to 0.0158, while turning off drift or bank lowers it to 0.0053 and 0.0000, respectively. The ablation table does not include a separate OursNoOverlap row because that configuration is already promoted into the main table. By contrast, OursFull (the overlap-enabled row) reaches seen_avg 0.0000 and token_f1 0.0000 in the current single-seed matrix. This means the bank/router stack still appears useful on seen-average, but anti-overlap is not yet a validated gain and should be treated as an unresolved risk.

Table 2. Key ablations on InstrDialog.

method	final_step	seen_avg	forgetting	token_f1	oracle_agreement
OursFull	0.0000	0.0000	0.1370	0.0000	0.0853
OursNoBank	0.0000	0.0000	0.0333	0.0063	0.0000
OursNoDrift	0.0000	0.0053	0.1546	0.0680	1.0000
OursNoRouter	0.0000	0.0158	0.1408	0.1091	0.0000

The current main-table Ours row on InstrDialog reports false_alarm_rate 0.0000, miss_rate 0.8889, and detection_delay_mean 9.5000. This indicates a conservative detector that is not firing often enough under the present proxy definition of shifts. Sequence-behavior diagnostics remain important. The latest diagnosis report attributes the dominant bottleneck to 'first-token margin too weak', with failure counts first_token_wrong=5, extra_preamble=2, and prefix_drift_1to5=1. Overfit exact match improved from 1/8 to 3/8 after removing duplicated BOS in generation-time prompt tokenization.

7.6 Current coverage gaps

- paper_instrdialogpp_router_only_s123 (instrdialog++ / router_only)

8. Current Contributions

- A unified, config-driven continual instruction tuning pipeline that covers baselines and the full bank/routing method inside one code path.
- A concrete implementation of core/probe anchor monitoring, LoRA bank management, router pseudo-label training, and orthogonal-weight anti-overlap regularization with activation-space monitoring.
- Script-generated tables, figures, and packaging outputs that can be refreshed without interrupting ongoing training runs.
- Additional behavior diagnostics that help separate continual-learning failures from open-loop generation failures.

9. Risks and Mitigations

- Risk 1: detector conservativeness. Current miss rate is high under the proxy shift definition. Mitigation: complete the planned threshold / anchor-size / monitor-interval sweeps and refine the proxy labeling scheme before making strong drift claims.

- Risk 2: router training / evaluation mismatch. The router is evaluated per example, but the active branch is still trained segment-wise. Mitigation: add a true routed training variant or clearly scope claims to inference-time routing only.
- Risk 3: forgetting remains weaker than replay-heavy baselines. Mitigation: treat seen-average and specialization gains as the current strength, not forgetting superiority, until more runs are complete.
- Risk 4: sequence-level behavior bottlenecks can contaminate continual-learning conclusions. Mitigation: keep the behavior gate and overfit diagnostics in the evaluation loop before promoting paper-grade claims.
- Risk 5: incomplete benchmark coverage and single-seed evidence. Mitigation: finish the missing InstrDialog++ runs and expand the validated winner to multi-seed evaluation before final packaging.

10. References

- Hu, E. J., et al. LoRA: Low-Rank Adaptation of Large Language Models.
- Wei, X., Li, G., and Marculescu, R. Online-LoRA: Task-free Online Continual Learning via Low Rank Adaptation.
- Zhang, Z., Fang, M., Chen, L., and Namazi-Rad, M.-R. CITB: A Benchmark for Continual Instruction Tuning.
- Kirkpatrick, J., et al. Overcoming catastrophic forgetting in neural networks.
- Liang, Y.-S., and Li, W.-J. GainLoRA: Gated Integration of Low-Rank Adaptation for Continual Learning of Language Models.
- Han, J., et al. SLIM: Let LLM Learn More and Forget Less with Soft LoRA and Identity Mixture.
- Page, E. S. CUSUM and sequential change-point monitoring.
- Soviany, P., Ionescu, R. T., Rota, P., and Sebe, N. Curriculum Learning: A Survey.
- Lopes, J. F., et al. Online Meta-Recommendation of CUSUM Hyperparameters.
- Kim, H. Geometric Regularization in Mixture-of-Experts: The Disconnect Between Weights and Activations.
- Mu, S., and Lin, S. A Comprehensive Survey of Mixture-of-Experts: Algorithms, Theory, and Applications.